



Chapter 1

Concepts for Object Databases

Advanced Database, Computer Science Dep't, DMU

Chapter Outline

- 1 Overview of O-O Concepts
- 2 O-O Identity, Object Structure and Type Constructors
- 3 Encapsulation of Operations, Methods and Persistence
- 4 Type and Class Hierarchies and Inheritance
- 5 Complex Objects
- 6 Other O-O Concepts
- 7 Summary

Introduction

Traditional data models (relational, network, and hierarchical) is successful in traditional database applications.

Shortcomings: engineering design, manufacturing, scientific experiments, telecommunications, geographic information systems and multimedia.

New requirements: complex structure for objects, longer-duration transactions, new data-types for storing images or large textual items, and the need to define non standard application-specific operations.

Introduction ...

Key feature of OODB: designer can specify both the structure of complex objects and the operations to them.

Goal of OODB:

maintain a direct correspondence between real-world and database objects so that objects do not lose their integrity and identity and can easily be identified and operated upon.

A decorative vertical strip on the left side of the slide. It features a green chalkboard texture. At the top, there is a dark, textured area. Below it, two pieces of pink chalk are shown, one standing upright and one lying horizontally. A white chalk line is drawn in a curved shape above the chalk, and a white arrow points upwards from the bottom left.

Overview of Object-Oriented Concepts

Overview...

Object:

Two component of an object:

state(value) and behavior(operations).

Types :

- ✓ **Transient objects:** exist only during program execution
- ✓ **Persistent object:** persist beyond program termination and can be retrieved later.

Overview...

In OO databases, objects may have an object structure of arbitrary complexity in order to contain all of the necessary information that describes the object.

In contrast, in traditional database systems, information about a complex object is often scattered over many relations or records, leading to loss of direct correspondence between a real world object and its database representation.

Overview...

The internal structure of an object in OOPLs includes the specification of **instance variables**, which hold the values that define the internal state of the object.

An instance variable is similar to the concept of an attribute, except that instance variables may be encapsulated within the object and thus are not necessarily visible to external users



Encapsulation of Operations, Methods and Persistence

Encapsulation of Operation, Methods

Main idea: define the behavior of a type of object based on the operations that can be **externally** applied to objects of that type.

Method: operation implementation
(interface: operation signature)

Complete encapsulation is stringent.

Visible attributes: may be directly accessed for reading by external operators, or by high level query language.

Encapsulation...

hidden attributes: completely encapsulated, can be accessed only through predefined operators.





Some OO models insist that all operations a user can apply to an object must be predefined. This forces a complete encapsulation of objects.

To encourage encapsulation, an operation is defined in two parts:

- ✓ **signature** or **interface** of the operation, specifies the **operation name and arguments** (or parameters).
- ✓ **method** or **body**, specifies the implementation of the operation.

Encapsulation...

Operations can be invoked by passing a message to an object, which includes the **operation name** and the **parameters**.

- ✓ The object then executes the method for that operation.

This encapsulation permits modification of the internal structure of an object, as well as the implementation of its operations, without the need to disturb the external programs that invoke these operations

Encapsulation...

Some OO systems provide capabilities for dealing with multiple versions of the same object (a feature that is essential in design and engineering applications).

- ✓ For example, an old version of an object that represents a tested and verified design should be retained until the new version is tested and verified:
- ✓ very crucial for designs in manufacturing process control, architecture , software systems



Operator polymorphism:

This refers to an operation's ability to be applied to different types of objects; in such a situation, an operation name may refer to several distinct implementations, depending on the type of objects it is applied to.

This feature is also called **operator overloading**

Persistence

Typical mechanism: naming and reachability

Naming: give an object a unique persistent name (must be unique within a particular database)

Reachability: works by making the object reachable from some persistent object.

A decorative background on the left side of the slide, featuring a green chalkboard texture. Two pieces of pink chalk are visible, one standing upright and one lying down. A white chalk arrow points upwards and to the right. The main title text is overlaid on this background.

Object Identity, Object Structure, and Type Constructors



Object Identity

Object Identity (OID): unique, system-generated, for each independent object, immutable (it's common to use long integers)

The main property required of an OID is that it be **immutable**; that is,

- ✓ the OID value of a particular object should not change.
- ✓ This preserves the identity of the real-world object being represented.

Object Structure

A feature of OODB: objects may have an object structure of arbitrary complexity.

The state of a complex object may be constructed from other objects.

Object triple representation: (i, c, v) i is a unique OID, c is a type constructor, v is the object state.



Type Constructors

Type Constructors: In OO databases, the state (current value) of a complex object may be constructed from other objects (or other values) by using certain type constructors.

The three most basic constructors are **atom**, **tuple**, and **set**. Other commonly used constructors include **list**, **bag**, and **array**.



The atom constructor is used to represent all basic atomic values, such as integers, real numbers, character strings, Booleans, and any other basic data types that the system supports directly.

The atom constructor is used to represent all basic atomic values: integers, real numbers, character strings, booleans.

Bag can have duplicate values.

Type Constructors

Basic constructors: atom, tuple, and set.

Others: list, bag, and array.

Represent object as a **graph structure**:
apply the type constructors recursively.

Identical states: Identical graph in every respect (OID at every level)

Equal states(shallow equality):

- ✓ Same graph structure
- ✓ Same corresponding atomic values
- ✓ Internal nodes may have different OIDs



Class Hierarchies and Inheritance

Class Hierarchies and Inheritance

Attributes and operations are together called functions (attributes resemble functions with zero arguments)

Type: a type name and a listing of the names of its visible (public) functions:

PERSON: Name, Address, Birthdate, Age, SSN

Defining a type:

- defining all functions

- Implementing them as attributes or as methods

extends

Extends: collections of objects of the same type

It's common in database application that each type or subtype will have an extent associated with it, which holds the collection of all persistent object of that type or subtype.

Persistent collection, transient collection



Persistent Collection:

This holds a collection of objects that is stored permanently in the database and hence can be accessed and shared by multiple programs

Transient Collection:

This exists temporarily during the execution of a program but is not kept when the program terminates



Complex Objects

Complex Objects

Two main types: structured and unstructured

Structured:

- ✓ made up of components
- ✓ Defined by applying available type constructors recursively

Unstructured: a data type requires a large amount of storage (image or large textual object)

ODBMS can be viewed as having a extensible type system.

DBMS internals provide the underlying storage and retrieval capabilities.



Complex Objects...

Unstructured complex object:

These is provided by a DBMS and permits the storage and retrieval of large objects that are needed by the database application.

Typical examples of such objects are bitmap images and long text strings (such as documents); they are also known as binary large objects, or BLOBs for short.

This has been the standard way by which Relational DBMSs have dealt with supporting complex objects, leaving the operations on those objects outside the RDBMS.

Complex Objects ...

Structured complex object:

This differs from an unstructured complex object in that the object's structure is defined by repeated application of the type constructors provided by the OODBMS.

Hence, the object structure is defined and known to the OODBMS.

The OODBMS also defines methods or operations on it.



Other O-O Concepts

Other Concepts

- ✓ Polymorphism: same name bound to different implementations
- ✓ Multiple inheritance
- ✓ Selective inheritance

A decorative background on the left side of the slide, featuring a green chalkboard texture. Two pieces of pink chalk are visible, one standing upright and one lying down. A white chalk arrow points upwards and to the right.

Polymorphism (Operator Overloading):

This concept allows the same **operator name or symbol** to be bound to two or more **different implementations** of the operator, depending on the type of objects to which the operator is applied

For example + can be:

- ✓ Addition in integers
- ✓ Concatenation in strings (of characters)

Multiple Inheritance and Selective Inheritance

Multiple inheritance in a type hierarchy occurs when a certain subtype T is a subtype of two (or more) types and hence inherits the functions (attributes and methods) of both supertypes.

For example, we may create a subtype `ENGINEERING_MANAGER` that is a subtype of both `MANAGER` and `ENGINEER`.

- ✓ This leads to the creation of a type lattice rather than a type hierarchy.



Versions and Configurations

Many database applications that use OO systems require the existence of several **versions** of the same object

There may be more than two **versions** of an object.

Configuration:

A **configuration** of the complex object is a collection consisting of one version of each module arranged in such a way that the module versions in the configuration are compatible and together form a valid version of the complex object.

A green chalkboard occupies the left side of the image. Two pieces of pink chalk are lying on the board. There are some white chalk markings, including a curved line and a straight line with a small hook at the end.

Summary

Summary

Object identity:

Objects have unique identities that are independent of their attribute values.

Type constructors:

Complex object structures can be constructed by recursively applying a set of basic constructors, such as tuple, set, list, and bag.

Encapsulation of operations:

Both the object structure and the operations that can be applied to objects are included in the object class definitions.

Summary

Programming language compatibility:

Both persistent and transient objects are handled uniformly. Objects are made persistent by being attached to a persistent collection.

Type hierarchies and inheritance:

Object types can be specified by using a type hierarchy, which allows the inheritance of both attributes and methods of previously defined types.

Extents:

All persistent objects of a particular type can be stored in an extent. Extents corresponding to a type hierarchy have set/subset constraints enforced on them.



Summary

Support for complex objects:

Both structured and unstructured complex objects can be stored and manipulated.

Polymorphism and operator overloading:

Operations and method names can be overloaded to apply to different object types with different implementations.

Versioning:

Some OO systems provide support for maintaining several versions of the same object.